Attestor

A cross-chain loan repayment history you cannot lie to — built on USC.

BUIDL CTC 2026 FALL · ETHEREUM SEPOLIA → CREDITCOIN CC3 TESTNET · GITHUB.COM/RICHARDREKI/ATTESTOR

THE PROBLEM

Creditcoin exists to make credit history verifiable. But the money that builds that history — stablecoin loans and repayments — lives on Ethereum. Bridging the *record* across chains usually means trusting a relayer's word that a repayment happened. A credit history is worthless if it can be forged.

THE SOLUTION

A borrower repays on Ethereum; an autonomous agent posts it to a credit history on Creditcoin — but the record is written **only** if an Attestcoin proof shows the repayment really happened. The agent proposes; the on-chain book disposes. **Neither the agent nor we can write a false entry.**

AAVE V3 · SEPOLIA

A stranger repays a debt. **Not our contract, not our borrower** — we cannot cause, populate or alter the `Repay` event.

OFF-CHAIN AGENT

Watches, applies a risk gate, waits ~8 min for attestation, fetches the proof, proposes a posting.

CREDITCOIN CC3

`AaveLoanBook` verifies the proof on the USC BlockProver precompile, re-derives the event from the attested receipt, writes the history.

Remove USC and there is no product: the book holds no trusted input of its own — no owner-submitted number, no oracle, no signed report. Every write is downstream of one precompile call, `0x…0FD2 . verify(…)`, returning without reverting.

Why the source matters more than the checks. A history you issue to yourself proves only that you can issue history to yourself. `AttestedLoanBook` — same precompile, same checks, but proving repayments to a contract we wrote — is kept alongside as the control. The contrast is the argument.

THE SECURITY MODEL — USC PROVES *LESS* THAN IT APPEARS

It proves a transaction was **included in a confirmed block** — not that it *succeeded*, touched the contract you care about, or was sent by who it claims. The book enforces, on-chain, every property the proof does not:

CHECK	WITHOUT IT
`status == 1`	a reverted repayment posts as if it paid
`chainId (resolved)`	a valid proof from the wrong chain is accepted
`emitter + topic, as a pair`	anyone deploys a contract emitting Aave's event and is believed
`borrower from the event`	credit is assigned to whoever relays the proof
`repayer == borrower`	one funded account mints standing for any address it likes
`freshness, in source blocks`	an attested repayment is postable forever
`consumed per log`	a transaction repaying several debts records only the first

Three of those are forced by reading a protocol we did not write. Aave's `repay` takes `amount = uint256 max` to mean "all of it", so the calldata does not say how much was repaid — only the event does. One transaction can repay several debts; a routed repayment on this pool was observed at log index 500. And a USC attestation carries no timestamp anywhere, so age is `latestAttestedHeight − provenHeight` and cannot honestly be quoted in seconds. **51 contract tests; 28 reject a forged or invalid input.**

WHAT WE FOUND THAT THE DOCS DON'T SAY — SEVEN UNDOCUMENTED PROTOCOL FACTS

1. **The precompile reverts on a bad proof; it does not return** `false` as the SDK documents — so `if (!verified)` is dead code.
2. **~8-minute latency is a real constraint:** a 32-block reorg window named only in an error string, plus ~39 blocks of attestation lag. We surface it, not hide it.
3. `chainKey` **is environment-scoped** — key 1 is Sepolia on testnet, Ethereum mainnet on mainnet. Bind to `chainId`, or a promotion silently changes trust.
4. **SDK and chain names differ:** the function is `verify` on-chain, not the SDK's `verifySingle`; ChainInfo is snake_case on-chain.
5. **The proof envelope is** `(uint8,bytes[])`**, not** `bytes[]` — and the SDK's own docs say the wrong one. A consumer written from the docs decodes nothing.
6. **The precompiles can't be fork-tested, and the failure is silent** — a call to the codeless address succeeds and returns nothing, passing the wrong assertion.
7. `get_chain_by_key` **returns one tuple, not two values** — a bug we shipped; every mocked test passed, it reverted every time on-chain. It's why we built a live-check layer.

Findings 5 and 7 make a documentation-following consumer fail outright. They are our ecosystem contribution as much as the product is.

LIVE & VERIFIABLE — NO KEY, NO GAS

reproduce	`node spike.mjs`
tests	`forge test` — 51 pass
live	`node tools/live-check.mjs`
agent	20 tests, tsc clean

MockUSD `0xCFd5E8e6…d0ef (Sepolia)`
LoanBook `0xe31906a2…5e9 (CC3) · LoanRepay 0x08F8b91A… (Sep)`
repay (Sep) `0x49592b0c…` post (CC3) `0x0f3d4ca0…`
BlockProver `0x…0FD2` ChainInfo `0x…0FD3`

HONEST STATUS & ROADMAP

> **The full loop is live, on-chain both sides, and closed.** A borrower repaid 250 mUSD on Sepolia (`0x49592b0c…`); the agent proved it and posted it to the loan book on Creditcoin (`0x0f3d4ca0…`), which reads `totalRepaid 250000000` for the borrower; then `CreditLine` read that attested history and the borrower drew a **real 125 mUSD undercollateralised loan** against it (`0xb62ffcff…`). Credit sized entirely by an unforgeable cross-chain record. Nothing is mocked — reproduce with `node tools/post-once.mjs`.

Next: the agent running continuously as a service · richer risk scoring over multi-loan history · batch proofs (the SDK supports up to 10) · mainnet with real USDC and a governance-set source binding.